دليل الفهم والشرح المعماري التفصيلي (بالعامية المصرية)

تطبيق مستحضرات التجميل (Avon Cosmetics App) الشرح الهندسي الكامل للكود، بنية Clean Architecture، ودورة عمل السلة والفاتورة

توضيح شامل لكل الملفات، الدوال، الحسابات الرياضية، ومعالجة الأخطاء والافتراضات الهندسية

تم إعداد هذا المستند لمساعدتك في فهم كل تفصيلة كود تم تطبيقها في المشروع بدون أي تعقيد. يغطي الشرح الملفات المضافة والمعدلة في السلة والفاتورة، تدفق عملية Add to Cart من زر الشاشة وصولاً للـ API، طريقة حساب الفاتورة (Subtotal, Shipping, Tax, Total)، مسار البيانات من السلة إلى الدفع، حل مشكلة الـ Rebuild والإنترنت في نسخة الـ APK، مع بيان دور كل كلاس في الـ Clean Architecture.

فرع العمل في (Git (Branch:
feature/cart-api-integration

مستودع المشروع (Repository):
https://github.com/Loayy1/Avon_app.git

نمط إدارة الحالة (State Management):
Flutter BLoC / Cubit (Pragmatic Clean Arch)

فحص الجودة والتحليل (Static Analysis):
0 Errors, 0 Warnings (PASSED)

1. الملفات المضافة والمعدلة في السلة والفاتورة (Cart & Invoice Files)

دي كل الملفات اللي اتعاملنا معاها عشان نربط الـ Cart ونظبط حسابات الفاتورة من ملف الـ Postman Collection، مع وظيفة كل ملف بالتحديد:

اسم الملف والمسار	نوع الملف	وظيفة الملف في المشروع
`lib/core/network/api_endpoints.dart`	Core Endpoint	متعرف جواه الثوابت الموحدة لعناوين الـ Cart: `cart`، `cartAdd`، `cartUpdate`، `cartRemove`.
`lib/features/cart/data/models/cart_item_model.dart`	Data Model	بيمثل شكل عنصر السلة، وفيه `fromJson` بتستقبل الـ JSON سواء كان راجع Flat أو Nested جواه أوبجكت منتج.
`lib/features/cart/data/datasources/cart_remote_data_source.dart`	Data Source	المسؤول الوحيد عن استدعاءات الشبكة للـ Cart عبر `DioHelper` (Get, Add, Update, Remove).
`lib/features/cart/domain/repositories/cart_repository.dart`	Domain Repo	العقد (Interface) اللي بيحدد العمليات المتاحة للسلة دون الارتباط بأي تفاصيل تخص Dio أو الشبكة.
`lib/features/cart/data/repositories/cart_repository_impl.dart`	Repo Impl	تنفيذ العقد، بيكلم الـ DataSource ويفحص النجاح ويحول البيانات لـ `List<CartItemModel>`.
`lib/views/pages/cart/cubit.dart`	Presentation Logic	إدارة حالة السلة: جلب العناصر، التعديل التفاؤلي الفوري، الحذف، وحساب الفاتورة (Subtotal, Tax, Shipping, Total).
`lib/views/pages/cart/states.dart`	Cubit States	حالات السلة: `CartLoadingState`، `CartSuccessState`، `CartFailedState`، وغيرها.
`lib/views/pages/cart/view.dart`	UI Screen	واجهة السلة: بتعرض الليستة بالـ `CustomItemCart`، وأزرار `+` و `-`، وكارت الفاتورة، وزرار الدفع.
`lib/views/pages/cart/checkout.dart`	UI Screen	شاشة إتمام الدفع (الفاتورة): استقبلت `subtotal` وبتحسب الإجمالي والضريبة والشحن ديناميكياً.
`lib/views/pages/home/customs_widget.dart`	UI Widget	عزل زرار الإضافة `_AddToCartButton` عشان يغير الـ Spinner بتاعه فقط وميعملش Rebuild للشاشة.
`lib/views/pages/home/view.dart`	UI Screen	إضافة خاصية `buildWhen` لمنع إعادة رسم الصفحة الرئيسية نهائياً وقت الضغط على زر السلة.

2. معمارية Clean Architecture المطبقة فعلياً في المشروع

المشروع متقسم بأسلوب **Feature-First Pragmatic Clean Architecture**، يعني كل ميزة (Feature) مستقلة ومتقسمة إلى 3 طبقات:

1. طبقة البيانات (Data Layer)

مسؤولة عن جلب البيانات وتحويلها من وإلى السيرفر والتخزين المحلي:

- **Models:** زي `CartItemModel` و `ProductModel`، بتحول الـ JSON لأوبجكت Dart.
- **DataSources:** زي `CartRemoteDataSourceImpl`، بتنفذ طلبات HTTP عبر `DioHelper`.
- **Repositories Impl:** زي `CartRepositoryImpl`، بتنسق بين الـ DataSources وبترجع بيانات معالجة للـ Domain.

2. طبقة منطق العمل (Domain Layer)

أنقى طبقة في المشروع، ملهاش علاقة بـ Flutter أو Dio أو JSON:

- **Repositories Interfaces:** زي `CartRepository` و `HomeRepository`. دي عبارة عن عقود (Abstract Classes) بتحدد أسماء الدوال اللي طبقات العرض محتاجاها بدون ما تهتم إزاي هتتجاب من السيرفر.

3. طبقة العرض وإدارة الحالة (Presentation Layer):

بتشمل الـ **Cubits** (زي `CartCubit` و `HomeCubit`) اللي بتستقبل أوامر المستخدم من الزراير، وتكلم الـ Repository، وتطلق **States**، والشاشات (Views & Widgets) اللي بتسمع للـ States وتعرض البيانات أو مؤشرات التحميل بدون أي كود شبكة أو Business Logic داخل شاشات الـ UI.

3. تتبع عملية Add to Cart خطوة بخطوة من الضغط للظهور في السلة

دي الرحلة الكاملة من لحظة ما المستخدم يضغط على الزرار الصغير فوق المنتج لحد ما يشوفه جواه السلة:

1. **المستخدم يدوس على الزرار في الصفحة الرئيسية:**
   الملف: `lib/views/pages/home/customs_widget.dart`
   الكلاس: `_AddToCartButton` ← الدالة: `onPressed` .
2. **تغيير حالة الزرار المحلية واستدعاء الـ Cubit:**
   الزرار بيعمل `setState(() => _isLoading = true)` عشان يظهر Spinner صغير جواه هو بس.
   وبينادي الميثود: `HomeCubit.get(context).addToCart(widget.productId)` .
3. **الـ Cubit يكلم الـ Repository:**
   الملف: `lib/views/pages/home/cubit.dart`
   الميثود `addToCart(int productId)` بتنادي: `repository.addToCart(productId: productId)` .
4. **الـ Repository تنادي الـ RemoteDataSource:**
   الملف: `lib/features/home/data/repositories/home_repository_impl.dart`
   الميثود: `addToCart({required int productId, int quantity = 1})`
   بتنادي: `remoteDataSource.addToCart(productId: productId, quantity: quantity)` .
5. **الـ DataSource يبعت الطلب عبر DioHelper:**
   الملف: `lib/features/home/data/datasources/home_remote_data_source.dart`
   بيعمل استدعاء:

```
DioHelper.sendData(ApiEndpoints.cartAdd, params: {"productId": productId, "quantity": quantity}, requiresAuth: true)
```

   .
   الطلب بيروح كـ `POST /api/Cart/add?productId=1&quantity=1` ومعه توكن الدخول في الـ Headers.
6. **السيرفر يرد بالنجاح والـ UI يتفاعل:**
   الـ Response بترجع بـ 200 OK، والزرار في `customs_widget.dart` بيرجع `_isLoading = false` ، وبيظهر للمستخدم `SnackBar` أخضر: *"Product added to cart successfully"* .
   سر عدم الـ Rebuild: الـ `HomeView` فيها `buildWhen: (prev, current) => current is HomeSuccessState || current is HomeLoadingState` ، فمش بتعيد رسم الشاشة نهائياً!
7. **ظهور المنتج في السلة عند فتحها:**
   الملف: `lib/views/pages/cart/view.dart`
   أول ما السلة تفتح، الـ `BlocProvider` بينادي `CartCubit()..getCart()` .
   الميثود بتنادي `CartRepositoryImpl.getCartItems()` ← `CartRemoteDataSourceImpl.getCart()` ← `GET /api/Cart` .
   البيانات بترجع وتتحول لـ `CartItemModel` ، والـ `CartCubit` بيطلق `CartSuccessState` ، والـ `ListView` تعرض المنتج فوراً.

4. كيف يتم حساب الكمية والسعر والـ Total ومصدر كل قيمة؟

كل الحسابات المالية بتتم ديناميكياً داخل `CartCubit` وبتتحدث فورياً في الواجهة:

القيمة	طريقة الحساب الرياضية	المصدر الأصلي للقيمة
سعر المنتج (Price)	قيمة فردية لكل قطعة (مثلاً 150.00$)	راجعة من السيرفر عبر حقل `price` في `CartItemModel` .
الكمية (Quantity)	قيمة عددية صحيحة (تزيد بـ `+` وتنقص بـ `-`)	راجعة مبدئياً من السيرفر، وتتعدل محلياً في الـ State عبر ميثود `updateQuantity` .

القيمة	طريقة الحساب الرياضية	المصدر الأصلي للقيمة
المجموع الفرعي (Subtotal)	`items.fold(0.0, (sum, i) ⇒ sum + (i.price * i.quantity))`	محسوبة ديناميكياً داخل الـ `CartCubit` بتجميع (سعر كل منتج × كميته).
الشحن (Shipping Fee)	`items.isEmpty ? 0.0 : 20.0`	منطق عمل محلي (Business Logic): 20 جنيه لو السلة فيها عناصر، و 0 لو فاضية.
الضريبة (Tax)	`subtotal * 0.14`	محسوبة محلياً بنسبة ضريبة القيمة المضافة الرسمية (14% VAT).
الإجمالي النهائي (Total)	`subtotal + shippingFee + tax`	مجموع (المجموع الفرعي + مصاريف الشحن + قيمة الضريبة).

5. مسار انتقال البيانات من Cart إلى Invoice (المصادر والتدفق)

❓ هل البيانات جاية من API ولا Local Storage ولا State موجودة في التطبيق؟

الإجابة الدقيقة بالتفصيل:

- **بيانات المنتجات والأسعار الأولية:** جاية من الـ **API** عبر طلب `GET /api/Cart` .
- **البيانات اللحظية أثناء الاستخدام:** موجودة في الـ **State الحية** للتطبيق جوه الـ `CartCubit` في المتغير `List<CartItemModel> items` .
- **حسابات الفاتورة (Subtotal, Shipping, Tax, Total):** بتتحسب **Live في الـ State** وليست مخزنة في قاعدة بيانات.
- **التخزين المحلي (Local Storage):** وظيفته الوحيدة حفظ `token` المستخدم في `CacheHelper` عشان يصرح بالطلبات، لكن الفاتورة نفسها مش متخزنة محلياً في الـ SharedPreferences.
- **طريقة الانتقال لشاشة الدفع (Invoice / Checkout):** بتتنقل كـ Parameter مباشر عبر دالة التنقل عند الضغط على زر Checkout:

```
// في كود CartView (lib/views/pages/cart/view.dart):
goTo(page: CheckoutView(subtotal: cubit.subtotal));
```

شاشة `CheckoutView` بتستقبل الـ `subtotal` وتعيد حساب الشحن والضريبة والإجمالي وتطبع الفاتورة النهائية للمستخدم.

6. دور كل Cubit و State و Repository و DataSource و Model في المشروع

1. الـ Models

- `CartItemModel` : تحويل JSON عنصر السلة لكائن Dart وحساب `subTotal` .
- `ProductModel` : تفكيك بيانات المنتجات (الاسم، السعر، الصورة، الوصف).
- `AuthResponseModel` : قراءة استجابة السيرفر، التوكن، وحالة النجاح والرسائل.

2. الـ DataSources

- `CartRemoteDataSource` : تنفيذ طلبات الـ HTTP المباشرة للسلة عبر Dio.
- `HomeRemoteDataSource` : طلبات جلب المنتجات وإضافة عنصر للسلة.
- `AuthRemoteDataSource` : طلبات التسجيل وتسجيل الدخول والتحقق واستعادة كلمة المرور.

3. الـ Repositories

- `CartRepository` و `CartRepositoryImpl` : وسيط البيانات للسلة، بيفحص كود الاستجابة ويحول البيانات لموديلات نظيفة.
- `HomeRepository` و `HomeRepositoryImpl` : عزل منطق جلب منتجات المتجر عن الشاشات.
- `AuthRepository` و `AuthRepositoryImpl` : وسيط المصادقة مع حفظ التوكن في `CacheHelper` .

4. الـ Cubits & States

- `CartCubit` : العقل المدبر للسلة، يدير الليستة والكميات والتحديث التفاؤلي وحساب الفاتورة.
- `CartStates` : تطلق إشعارات للـ UI: تحميل، نجاح، فشل، تعديل، وحذف.
- `HomeCubit` : إدارة منتجات الشاشة الرئيسية وحالات التحميل والنجاح.
- `LogInCubit` / `RegisterCubit` : إدارة فورم التسجيل والتحقق.

7. معالجة الأخطاء وحالات Loading و Success و Failure

كيف يتعامل التطبيق مع الأخطاء والحالات المختلفة؟

- **أخطاء الشبكة والاتصال في `DioHelper` :** تم بناء ميثود `_handleDioError` اللي بتفحص نوع الـ `DioException` : لو مهلة الاتصال انتهت (Timeout) بترجع: *"Connection timed out. Please check your internet connection"*. ولو السيرفر غير متاح أو مفيش نت بترجع رسالة واضحة، ولو السيرفر رجع خطأ 400 أو 422 بتستخرج نص الخطأ من حقول `message` أو `errors` في الـ JSON.
- **حالات التحميل (Loading States):** إطلاق `LoadingState` في الكيوبت، وده بيخلي الـ UI يعرض مؤشر تحميل (CircularProgressIndicator) سواء في الزرار أو في نص الشاشة لمنع المستخدم من تكرار الضغط.
- **حالات النجاح (Success States):** إطلاق `SuccessState` مع تحديث البيانات المعروضة في الشاشة أو الانتقال للشاشة التالية بسلاسة.
- **حالات الفشل (Failure States):** إطلاق `FailedState(message: ...)` ، والـ `BlocConsumer` أو `BlocListener` بيستقبل الرسالة ويعرضها في `SnackBar` أحمر منسق وواضح للمستخدم دون أن ينهار التطبيق (Crash-Free).

8. الافتراضات الهندسية والمشاكل المحتملة في الكود (ملاحظات هندسية)

نقاط وافتراضات يجب الانتباه لها مستقبلاً (من غير تعديل الكود):

1. **قيم الشحن والضريبة الثابتة (Hardcoded Values):** مصاريف الشحن (20.0) ونسبة الضريبة (14%) مكتوبة كقيم ثابتة في الكيوبت وشاشة الـ Checkout. مستقبلاً لو السيرفر بيحسب الشحن حسب عنوان العميل أو المنطقة، يفضل إن القيمة تيجي مباشرة من الـ API.
2. **تنوع استجابة الـ Cart من السيرفر:** السيرفر أحياناً بيرجع السلة كـ Array مباشر `[{ ... }]` وأحياناً كـ Object جواه مفتاح `items` . الكود الحالي في `CartItemModel` بيتعامل بمرونة مع الشكلين، لكن لو السيرفر غير اسم المفتاح في المستقبل قد يتطلب تحديث الـ parser.
3. **مزامنة السلة بين الصفحات (Realtime Sync):** عند إضافة منتج من الهوم، بيتبعت للسيرفر، ولما تفتح شاشة الكارت بتعمل `getCart()` فبيظهر. لكن لو شاشة الكارت كانت مفتوحة أصلاً في الـ Stack في الخلفية، مش هتتحدث تلقائياً غير لما تعمل سحب لأسفل (Pull to refresh) لأن مفيش Socket أو Global Event Bus يربط بينهم لحظياً.
4. **تجديد التوكن التلقائي (Auto Token Refresh):** التوكن بيتخزن في `CacheHelper` ، لكن لو انتهت صلاحيته ورجع السيرفر خطأ `401 Unauthorized` ، مفيش Interceptor آلي يطلب Refresh Token في الخلفية، بل بيطلب من المستخدم تسجيل الدخول مرة تانية.

9. ملفات أندرويد وحل مشكلة الإنترنت في نسخة الـ APK

سبب المشكلة وحلها النهائي:

ملف `android/app/src/main/AndroidManifest.xml` مكانش فيه تصريح `INTERNET` لأن فلاتر بتحطه افتراضياً في فولدر `debug` بس. لما بتعمل APK للإنتاج، نظام أندرويد بيحجب أي اتصال خارجي برمز خطأ `Permission denied` .

الحل: تم وضع تصاريح `INTERNET` و `ACCESS_NETWORK_STATE` في الـ Manifest الرئيسي، وإنشاء ملف `network_security_config.xml` لتمكين حركة الشبكة وتوثيق الشهادات، وأصبح الـ APK يعمل بكفاءة تامة.